
순서 논리 회로;
Sequential Logic Circuit;
조합논리회로 + 기억소자로 구성

순서(順序); order, sequence;
정해진 기준에서 말하는 '전후' 같은 '차례' 관계

그럼, 순서를 알려면 뭐가 있어야 하나요?
바로, 이전 상태, 현재 상태와 같은
상태(State)을 기억할 수 있는 소자가 필요함

메모리(Memory) >> 1bit 저장할 수 있는 플립플롭(Flip-Flop)
순서를 판단하는 기준; 클럭(CP; Clock Pulse)

04 순서 논리 회로

1 순서 논리 회로의 개요

- 조합 논리 회로(combinational logic circuit) : 이전 입력 값에 관계없이 현재 입력 값에 따라 출력이 결정
- 순서 논리 회로(sequential logic circuit) : 현재의 입력 값과 이전 출력 상태에 따라 출력 값이 결정

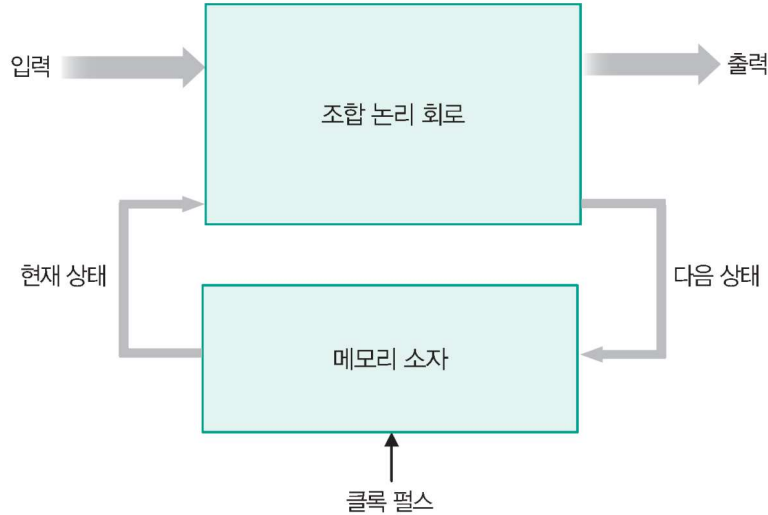


그림 3-62 순서 논리 회로의 구성

04 순서 논리 회로

❖ 순서 논리 회로의 특징

- 순서 논리 회로의 출력은 외부에서 들어온 입력과 이전 출력 상태에 따라 결정된다.
- 이러한 동작은 클록 펄스가 들어올 때마다 반복해서 일어난다.
- 순서 논리 회로는 기억 기능(플립플롭)이 있다.
- 대표적인 순서 논리 회로에는 플립플롭, 카운터, 레지스터 등이 있다.

□ 클록 펄스

- 플립플롭(flip-flop)은 클록 펄스(Clock Pulse, CP)라는 제어 입력을 가지며, 출력은 클록 펄스에 동기되어 변하고 이러한 변화를 트리거(trigger)되었다고 한다.

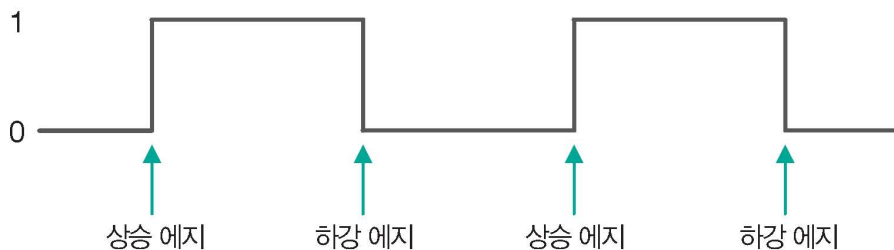
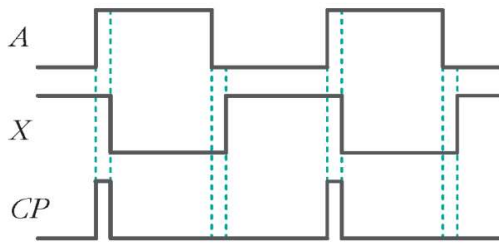
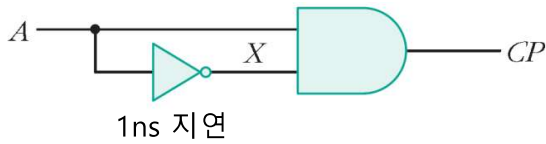


그림 3-63 클록 펄스의 형태

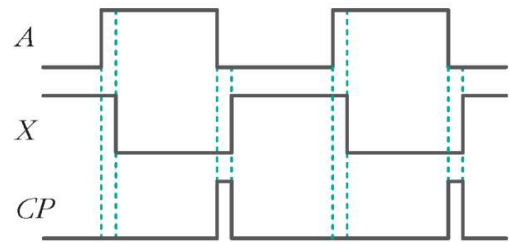
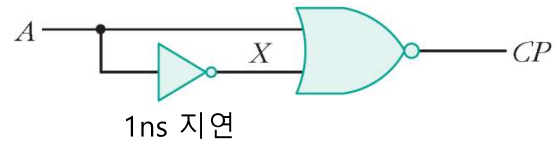
04 순서 논리 회로

❖ 펄스 전이 검출기

- 실제 회로에서 클럭 펄스는 상승 에지나 하강 에지에서 순간적으로 1이 되는 날카로운 파형을 만들어 플립플롭을 동작시키는데, 이 파형을 에지 트리거(edge trigger)라고 한다.
- 클럭 펄스(구형파)로 에지 트리거를 만들려면 **펄스 전이 검출기**가 필요하다



(a) 상승 에지 트리거



(b) 하강 에지 트리거

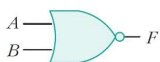
그림 3-64 펄스 전이 검출기 구조

04 순서 논리 회로

SR 래치

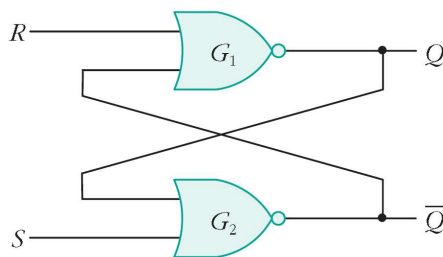
S(t+1) Q(t) Q'(t+1)

입력	출력
A B	F
0 0	1
0 1	0
1 0	0
1 1	0



(a) 2입력인 경우: $F = \overline{A+B}$

그림 3-8 NOR 게이트의 진리표와 논리 기호



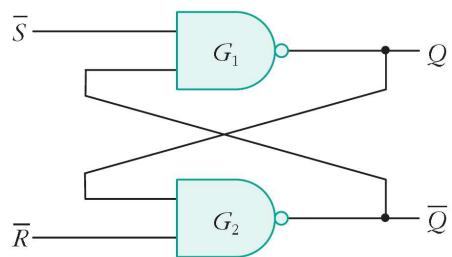
S(t+1)	R(t+1)	Q(t+1)
S	R	
0	0	Q(t)(불변)
0	1	0
1	0	1
1	1	부정

(a) NOR 게이트로 구성

그림 3-66 SR 래치의 논리 회로와 진리표

[SR 래치]

- 플립플롭의 기본이 되는 조합논리회로
- (시간 t) 출력을 다시 입력으로 이용하여 (시간 t+1) 출력을 만드는 원리

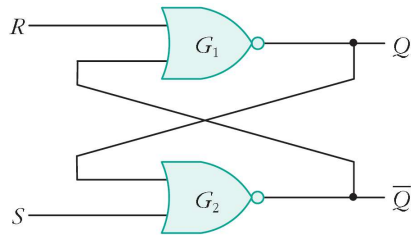


S	R	Q(t+1)
S	R	
0	0	부정
0	1	1
1	0	0
1	1	Q(t)(불변)

(b) NAND 게이트로 구성

SR이 모두 1되면 출력이 부정이 되는 것을 해결하기 위해서, 다양한 플립플롭이 등장하게 됨

SR 래치 파형



S	R	Q(t+1)
0	0	Q(t) (불변)
0	1	0
1	0	1
1	1	부정

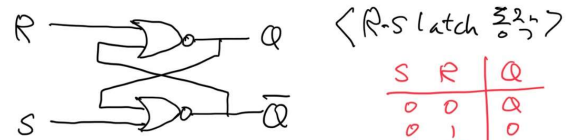
(a) NOR 게이트로 구성

그림 3-66 SR 래치의 논리 회로와 진리표

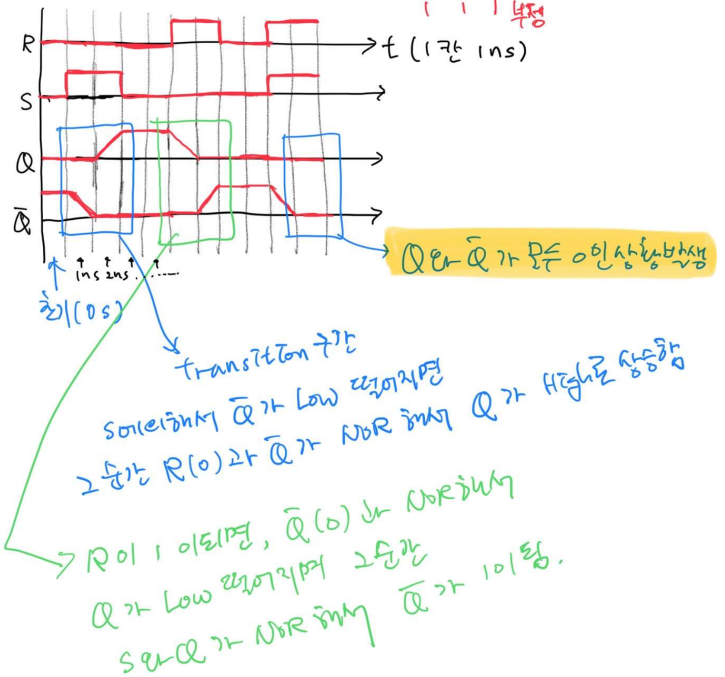
입력	출력
A B	F
0 0	1
0 1	0
1 0	0
1 1	0

(a) 2입력인 경우: $F = \overline{A+B}$

그림 3-8 NOR 게이트의 진리표와 논리 기호



S	R	Q
0	0	Q
0	1	0
1	0	1
1	1	부정



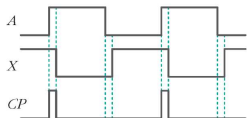
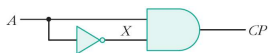
04 순서 논리 회로

플립플롭의 종류

래치+CP동기화 → 플립플롭

플립플롭의 특징

- 플립플롭은 1비트의 정보를 기억할 수 있는 기억 소자다.
- 플립플롭은 제어 입력인 클록 펄스(CP)가 있으며 다음 클록 펄스(CP)가 들어올 때까지 현재 상태를 유지한다.
- 플립플롭은 Q와 Q-bar로 표시된 출력이 2개 있으며 Q와 Q-bar의 상태는 서로 보수가 되어야 정상 상태가 된다.
- 플립플롭은 메모리의 구성 요소로도 사용된다.
- 플립플롭에는 SR 플립플롭, JK 플립플롭, D 플립플롭, T 플립플롭이 있다.



(a) 상승 에지 트리거

그림 3-64 펄스 전이 검출기 구조

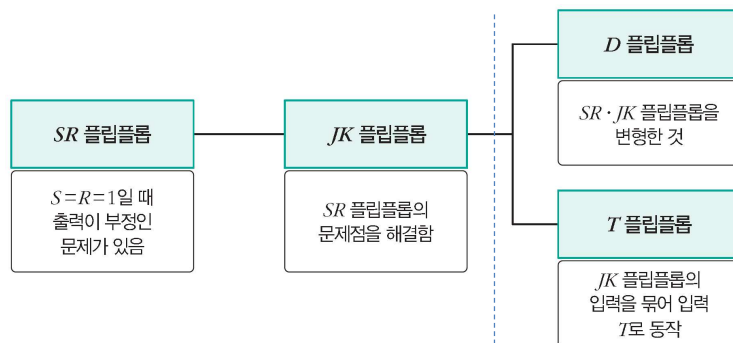


그림 3-65 플립플롭의 종류

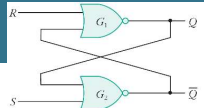
입력 2개

입력 1개

04 순서 논리 회로

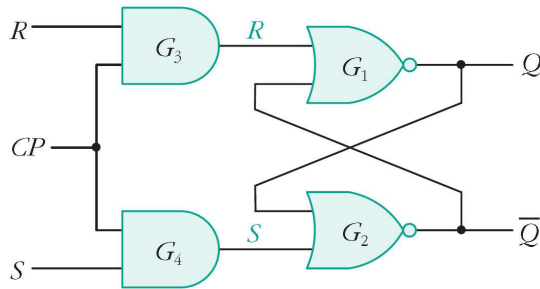
□ SR 플립플롭

- SR 플립플롭 : 클록 펄스가 있을 때만 동작하는 SR 래치를 의미
- 클록 펄스는 **상승 에지 트리거 신호**가 입력된 경우다.

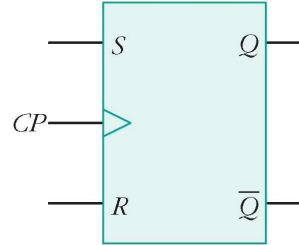


S	R	Q(t+1)
0	0	Q(t)(불변)
0	1	0
1	0	1
1	1	부정

(a) NOR 게이트로 구성
그림 3-66 SR 래치의 논리 회로와 진리표



(a) 논리 회로



(b) 논리 기호

S	R	Q(t+1)
0	0	Q(t)(불변)
0	1	0
1	0	1
1	1	부정

(c) 진리표

그림 3-67 SR 플립플롭의 구조

CP=0인 경우	S와 R의 입력에 관계없이 앞단의 AND 게이트 G ₃ 과 G ₄ 의 출력이 항상 0이므로 플립플롭의 출력은 불변
CP=1인 경우	S와 R의 입력이 회로 후단의 NOR 게이트 G ₁ 과 G ₂ 의 입력으로 전달되어 SR 래치와 같은 동작을 수행

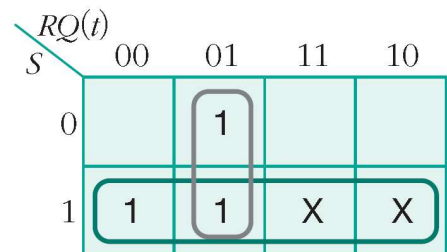
04 순서 논리 회로

❖ SR 플립플롭의 특성표(진리표) 및 특성 방정식(부울 대수식)

S	R	Q(t)	Q(t+1)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	부정
1	1	1	부정

(a) 특성표

S	R	Q(t+1)
0	0	Q(t)(불변)
0	1	0
1	0	1
1	1	부정



$$Q(t+1) = S + \bar{R}Q(t), SR = 0$$

(b) 특성 방정식

그림 3-68 SR 플립플롭의 특성표와 특성 방정식

SR=0: S와 R이 모두 1인 경우는 없어야 한다는 제약사항

04 순서 논리 회로

□ D 플립플롭

- SR 플립플롭에서 원하지 않는 상태($S=R=1$)를 제거하는 한 가지 방법
- SR 플립플롭을 변형한 것
- 입력신호 D 가 CP 에 동기되어 그대로 출력에 전달되는 특성을 가지고 있음
- D 플립플롭이라는 이름은 데이터(Data)를 전달하는 것과 지연(Delay)하는 역할에서 유래

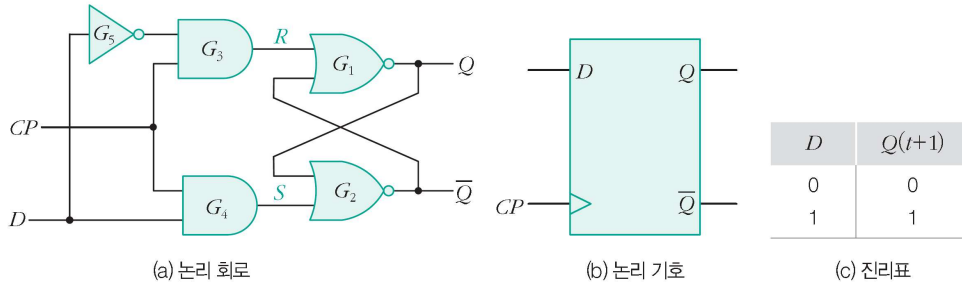


그림 3-69 D 플립플롭의 구조

$CP=1, D=1$	G_3 의 출력은 0, G_4 의 출력은 1이 된다. 따라서 SR 래치의 입력은 $S=1, R=0$ 이 되므로 결과적으로 $Q=1$ 을 얻는다.
$CP=1, D=0$	G_3 의 출력은 1, G_4 의 출력은 0이 된다. 따라서 SR 래치의 입력은 $S=0, R=1$ 이 되므로 결과적으로 $Q=0$ 을 얻는다.

S	R	$Q(t+1)$
0	0	$Q(t)$ (불변)
0	1	0
1	0	1
1	1	부정

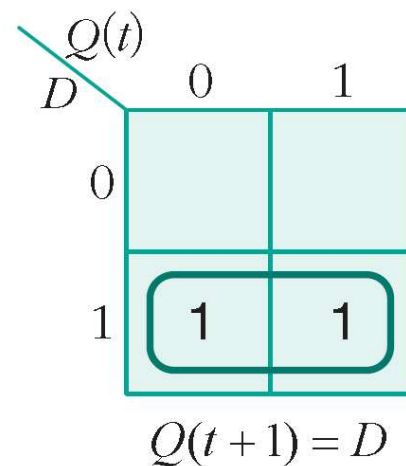
**CP=0인 경우,
R=0, S=0 → Q 불변 → 1비트 저장**

04 순서 논리 회로

❖ D 플립플롭의 특성표 및 특성 방정식

D	$Q(t)$	$Q(t+1)$
0	0	0
0	1	0
1	0	1
1	1	1

(a) 특성표



(b) 특성 방정식

그림 3-70 D 플립플롭의 특성표와 특성 방정식

□ JK 플립플롭

- JK 플립플롭은 SR 플립플롭에서 $S=1, R=1$ 인 경우 출력이 불안정한 상태가 되는 문제점을 개선하여 $S=1, R=1$ 에서도 동작하도록 개선한 회로
- JK 플립플롭의 J는 S(set)에, K는 R(reset)에 대응하는 입력
- $J=1, K=1$ 인 경우 JK 플립플롭의 출력은 이전 출력의 보수 상태(반전)로 변화
- JK 플립플롭은 플립플롭 중에서 가장 많이 사용되는 플립플롭이다.

□ JK 플립플롭

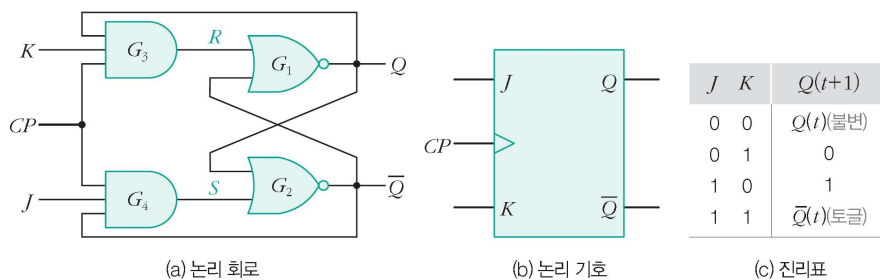


그림 3-71 JK 플립플롭의 구조

**J=1, K=1 인 경우에
CP가 1일 때
Q가 토글(반전) 됨**

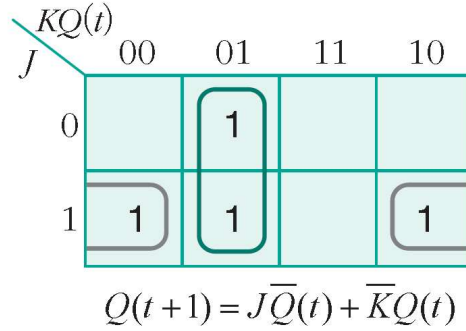
$J=0, K=0$	G_3 과 G_4 의 출력이 모두 0이므로 G_1 과 G_2 로 구성된 SR 래치는 출력이 변하지 않는다.
$J=0, K=1$	G_4 의 출력은 0이 되고 G_3 의 출력은 $Q(t) \cdot K \cdot CP$ 인데 $K=1, CP=1$ 이므로 $Q(t)$ 가 된다. $Q(t+1)=0$
$J=1, K=0$	G_3 의 출력은 0이 되고 G_4 의 출력은 $\bar{Q}(t) \cdot J \cdot CP$ 인데 $J=1, CP=1$ 이므로 $\bar{Q}(t)$ 가 된다. $Q(t+1)=0, Q(t+1)=1$
$J=1, K=1$	G_3 의 출력은 $Q(t) \cdot K \cdot CP$ 인데 $K=1, CP=1$ 이므로 $Q(t)$ 가 된다. 또한 G_4 의 출력은 $\bar{Q}(t) \cdot J \cdot CP$ 인데 $J=1, CP=1$ 이므로 $\bar{Q}(t)$ 가 된다. $Q(t)=0$ 인 경우 SR 래치의 $S=1, R=0$ 인 경우와 같으므로 출력은 $Q(t+1)=1$ 이 된다. 마찬가지로 $Q(t)=1$ 인 경우 SR 래치의 $S=0, R=1$ 인 경우와 같으므로 출력은 $Q(t+1)=0$ 이 된다. 따라서 출력은 보수가 된다.

04 순서 논리 회로

❖ JK 플립플롭의 특성표 및 특성 방정식

J	K	$Q(t)$	$Q(t+1)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

(a) 특성표



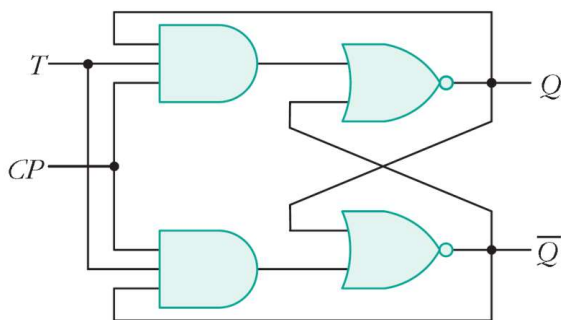
(b) 특성 방정식

그림 3-72 JK 플립플롭의 특성표와 특성 방정식

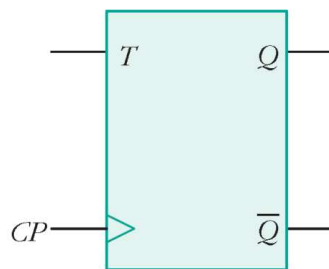
04 순서 논리 회로

□ T 플립플롭 (토글 toggle 플립플롭)

- JK 플립플롭의 J 와 K 입력을 묶어서 하나의 입력신호 T 로 동작시키는 플립플롭
- T 플립플롭의 입력 $T=0$ 이면, T 플립플롭은 $J=0, K=0$ 인 JK 플립플롭과 같이 동작하므로 출력은 변하지 않는다. $T=1$ 이면, $J=1, K=1$ 인 JK 플립플롭과 같이 동작하므로 출력은 보수가 된다.



(a) 논리 회로



(b) 논리 기호

T	$Q(t+1)$
0	$Q(t)$
1	$\bar{Q}(t)$

(c) 진리표

그림 3-73 T 플립플롭

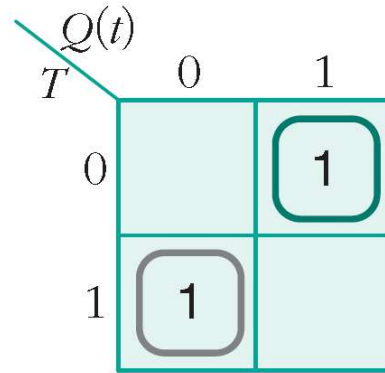
04 순서 논리 회로

❖ T 플립플롭의 특성표 및 특성 방정식

T	$Q(t)$	$Q(t+1)$
0	0	0
0	1	1
1	0	1
1	1	0

T가 0이면
변화 없음.
1이면 반전

(a) 특성표



$$Q(t+1) = T\bar{Q}(t) + \bar{T}Q(t)$$

(b) 특성 방정식

그림 3-74 T 플립플롭의 특성표와 특성 방정식

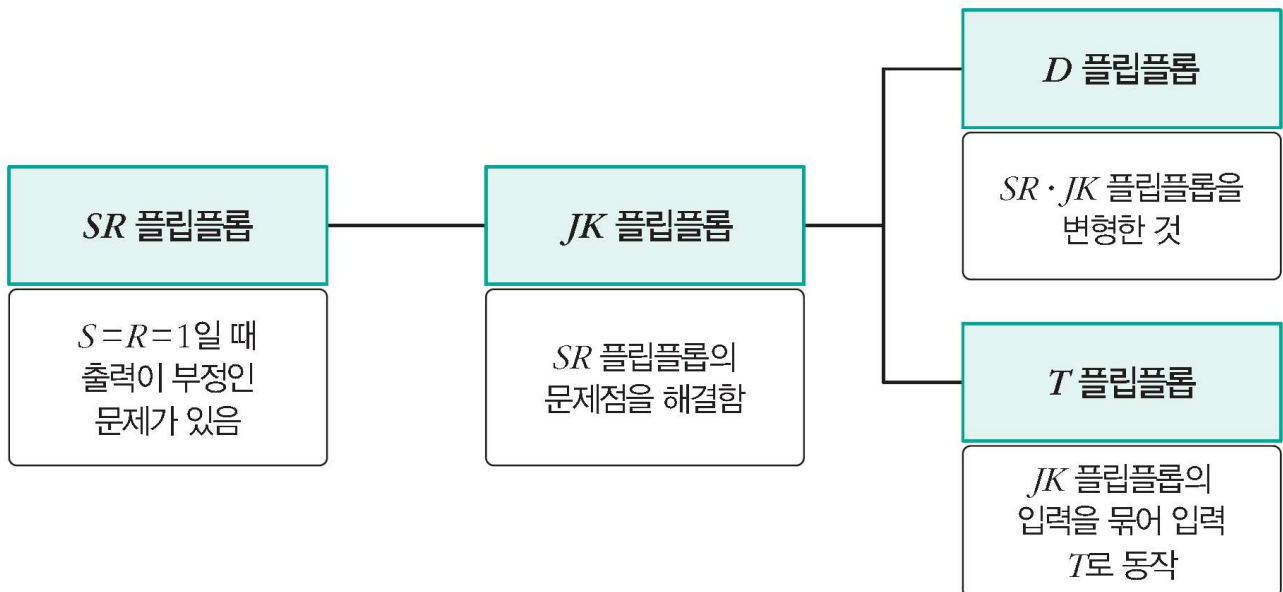


그림 3-65 플립플롭의 종류

04 순서 논리 회로

$J=1, K=1$ 인 경우에 CP 가 1인 동안 Q 가 계속 토글(반전) 되는 문제

□ 주종형 JK 플립플롭

- 레이스 현상 문제(racing problem) : JK 플립플롭은 $J = K = 1$ 인 경우 클록 펄스가 길어지면 출력이 계속 반전되는 현상
- 이를 해결하는 방법은 에지 트리거를 이용하거나 주종형 JK 플립플롭(master-slave JK flip-flop)을 사용하는 것이다.

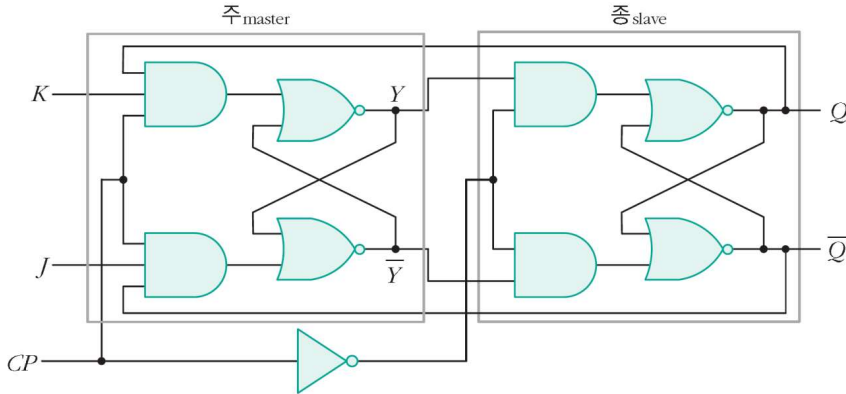


그림 3-75 주종형 JK 플립플롭

$CP=1$	외부의 J 와 K 의 입력이 Master 플립플롭에 전달 Slave 플립플롭은 $CP=0$ 이므로 동작하지 않음
$CP=0$	Slave 플립플롭이 동작하여 $Q = Y, \bar{Q} = \bar{Y}$ Master 플립플롭은 $CP=0$ 이므로 동작하지 않음

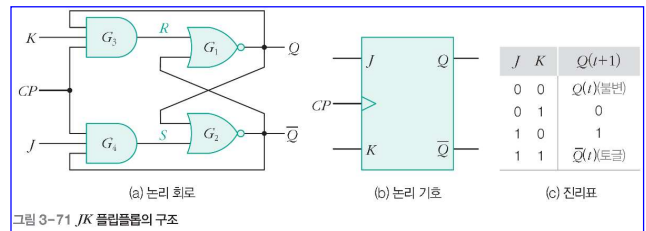
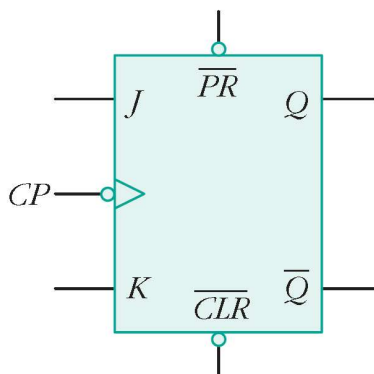


그림 3-71 JK 플립플롭의 구조

04 순서 논리 회로

□ 비동기 입력

- 대부분의 플립플롭은 클록펄스에 의해서 플립플롭의 상태를 변화시킬 수 있는 동기입력이 있고, 클록펄스와 관계없이 비동기적으로 변화시킬 수 있는 비동기 입력인 preset($\overline{PR}$) 입력과 clear($\overline{CLR}$) 입력이 있다.
- 비동기 입력들은 플립플롭의 초기조건을 결정하는 등 다방면으로 유용하게 사용



(a) 논리 기호

$\overline{PR}$	$\overline{CLR}$	CP	J	K	$Q(t+1)$
0	1	×	×	×	1
1	0	×	×	×	0
1	1	↓	0	0	변화 없음
1	1	↓	0	1	0
1	1	↓	1	0	1
1	1	↓	1	1	토글

(b) 진리표

그림 3-76 비동기 입력을 가진 JK 플립플롭의 논리 기호와 진리표

04 순서 논리 회로

3 순서 논리 회로의 설계

□ 여기표

- 플립플롭의 특성표 : 현재 상태와 입력값이 주어졌을 때, 다음 상태가 어떻게 변하는가를 나타내는 표
- 플립플롭의 여기표(excitation table) : 현재 상태에서 다음 상태로 변했을 때 플립플롭의 입력조건이 어떤 상태인가를 나타내는 표
- 플립플롭의 여기표는 순서논리회로를 설계할 때 자주 사용

진리표:
입력이 주어지고, 출력을 정함

여기표:
출력이 주어지고, 입력을 유추함

04 순서 논리 회로

❖ SR 플립플롭의 여기표

- ① $Q(t)=0$ 에서 $Q(t+1)=0$ 으로 변하는 경우는 $S=0, R=0$ 과 $S=0, R=1$ 인 경우 두 가지다. 이것은 $S=0, R=\times$ 로 표시한다.
- ② $Q(t)=0$ 에서 $Q(t+1)=1$ 로 변하는 경우는 $S=1, R=0$ 인 경우만 변화가 가능하다.
- ③ $Q(t)=1$ 에서 $Q(t+1)=0$ 으로 변하는 경우는 $S=0, R=1$ 인 경우만 변화가 가능하다.
- ④ $Q(t)=1$ 에서 $Q(t+1)=1$ 로 변하는 경우는 $S=0, R=0$ 과 $S=1, R=0$ 인 경우다. 이것은 $S=\times, R=0$ 으로 표시한다.

특성표			여기표		
입력 $S \quad R$	현재 상태 $Q(t)$	다음 상태 $Q(t+1)$	현재 상태 $Q(t)$	다음 상태 $Q(t+1)$	요구 입력 $S \quad R$
0 0	0	0	0	0	0 ×
0 0	1	1	0	1	1 0
0 1	0	0	1	0	0 1
0 1	1	0	1	1	× 0
1 0	0	1			
1 0	1	1			
1 1	0	?			
1 1	1	?			

SR 플립플롭의 진리표		
$S \quad R$	$Q(t+1)$	
0 0	$Q(t)$ (불변)	
0 1	0	
1 0	1	
1 1	부정	

그림 3-77 SR 플립플롭의 여기표 유도 과정

❖ JK 플립플롭의 여기표

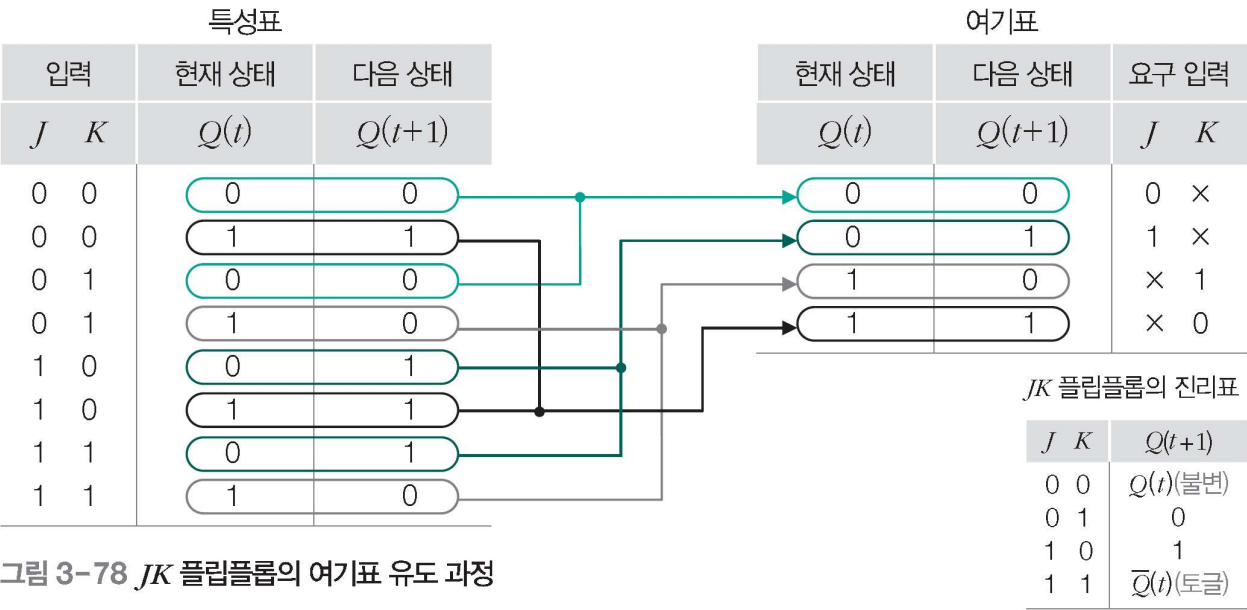


그림 3-78 JK 플립플롭의 여기표 유도 과정

❖ D 플립플롭의 여기표

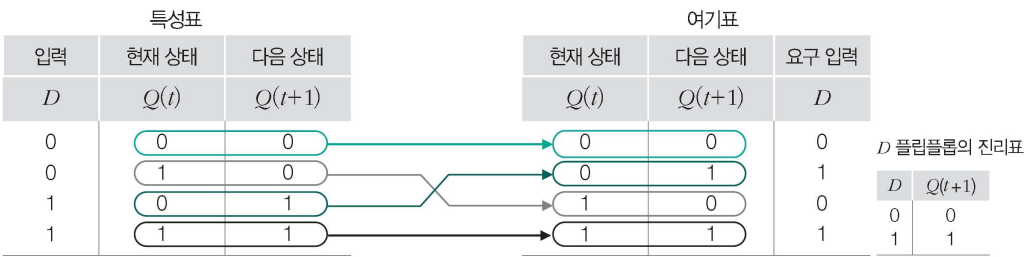


그림 3-79 D 플립플롭의 여기표 유도 과정

❖ T 플립플롭의 여기표

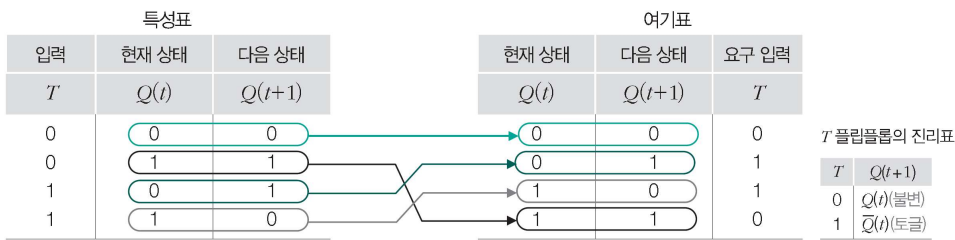


그림 3-80 T 플립플롭의 여기표 유도 과정

04 순서 논리 회로

□ 순서 논리 회로의 설계 과정

- ① 설계 사양으로부터 상태도와 상태표 작성
- ② 플립플롭의 수와 종류 결정
- ③ 플립플롭의 입력, 출력 및 각 상태에 문자 기호 부여
- ④ 상태표를 이용해 회로의 상태 여기표 작성
- ⑤ 간소화 방법을 이용해 출력 함수와 플립플롭의 입력 함수 유도
- ⑥ 순서 논리 회로도 작성

04 순서 논리 회로

① 설계 사양으로부터 상태도(State Diagram)와 상태표(State Table) 작성

출력 AB가 00 상태에서
입력 x가 1이면(1/), 01
상태로 변함

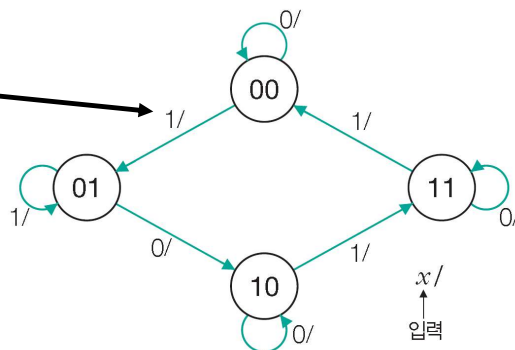
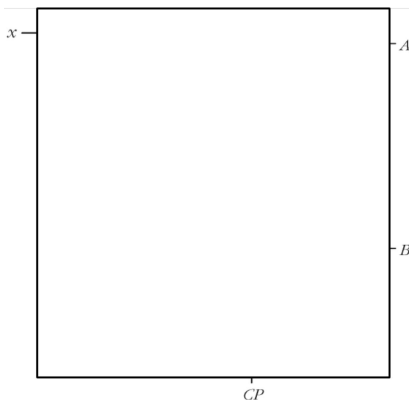


그림 3-81 순서 논리 회로에 대한 상태도

표 3-8 그림 3-81의 상태표

현재 상태		입력 x	다음 상태	
A	B		A	B
0	0	0	0	0
0	0	1	0	1
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	1	1
1	1	0	1	1
1	1	1	0	0

② 플립플롭의 수와 종류 결정하고 ③ 각 상태에 문자 기호 부여

- 네 가지 상태가 있으므로 플립플롭이 2개(2비트 기억) 필요하며, 각 플립플롭에 문자 A 와 B 를 할당한다.
- JK 플립플롭을 이용한다.

n 개의 서로 다른 상태를 나타내려면 플립플롭이 $\lceil \log_2 n \rceil$ 개 필요하다. 예를 들어 $n=100$ 이면 $\log_2 100 \approx 3.219$ 이므로 플립플롭이 $\lceil \log_2 100 \rceil = 4$ 개 필요하다.

상태도 (State Diagram)

어떤 상태에서 어떤 입/출력인 경우 어떤 상태로 변화되는 순서도

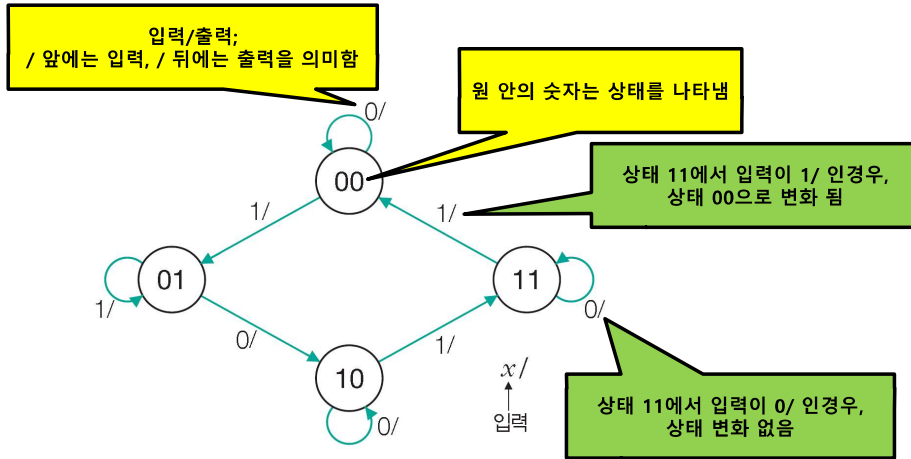


그림 3-81 순서 논리 회로에 대한 상태도

*위의 예제는 별도의 제어출력이 없음. (제어입력 x는 있음)
단지, 상태출력(기억) A, B만 있는 경우임

표 3-8 그림 3-81의 상태표

현재 상태		입력	다음 상태	
A	B	x	A	B
0	0	0	0	0
0	0	1	0	1
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	1	1
1	1	0	1	1
1	1	1	0	0

필요한 플립플롭 수와 종류 결정하기

순서논리회로는 조합논리회로와 기억소자로 구성됨

- AND, OR 같은 기본 논리 게이트의 조합과 플립플롭 1개 이상 병렬 또는 직렬로 결합함

기억해야할 상태의 수(=상태의 경우의 수; 상태를 나타내는 비트로 표현할 수 있는 수)

- 상태의 수가 4개면; 상태의 경우의 수가 4개
- 상태를 나타내기 위해서 2비트가 필요함 → 2비트; 플립플롭 2개가 필요하다는 것
- k비트로 표현할 수 있는 상태의 수: $n = 2^k$,
- n개의 상태를 나타내기 위한 비트의 수: $k = \log_2 n$ ← k가 필요한 플립플롭의 개수

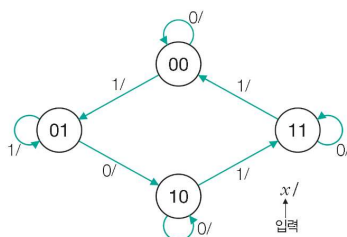


그림 3-81 순서 논리 회로에 대한 상태도

4가지 상태 == 2비트 필요
== 플립플롭 2개 필요

10가지 상태라면,
 $n = 10, k = \log_2 10 = 3.219 < 4$
4 비트 필요 == 플립플롭 4개 필요

04 순서 논리 회로

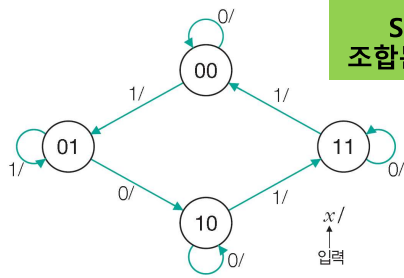


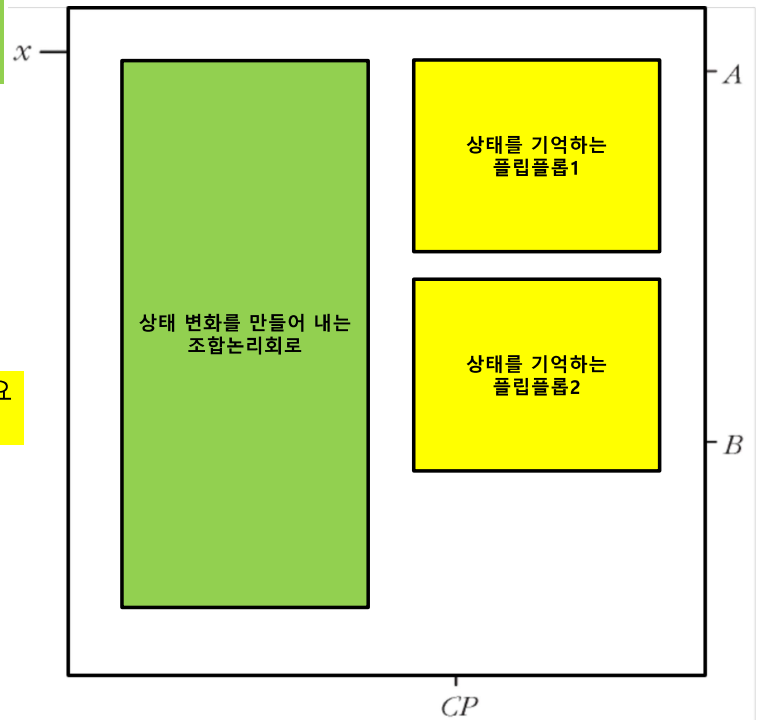
그림 3-81 순서 논리 회로에 대한 상태도

표 3-8 그림 3-81의 상태표

현재 상태		입력	다음 상태	
A	B	x	A	B
0	0	0	0	0
0	0	1	0	1
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	1	1
1	1	0	1	1
1	1	1	0	0

4가지 상태 == 2비트 필요
== 플립플롭 2개 필요

순서 논리 회로;
Sequential Logic Circuit;
조합논리회로 + 기억소자로 구성



04 순서 논리 회로

● 상태표를 이용해 회로의 상태 여기표 작성

표 3-9 상태 여기표

상태A가 0 → 0 이 되려면, J_A는?
상태B가 0 → 1 이 되려면, J_B는?

현재 상태			입력	다음 상태		플립플롭 입력			
A	B	x		A	B	J _A	K _A	J _B	K _B
0	0	0		0	0	0	×	0	×
0	0	1		0	1	0	×	1	×
0	1	0		1	0	1	×	×	1
0	1	1		0	1	0	×	×	0
1	0	0		1	0	×	0	0	×
1	0	1		1	1	×	0	1	×
1	1	0		1	1	×	0	×	0
1	1	1		0	0	×	1	×	1

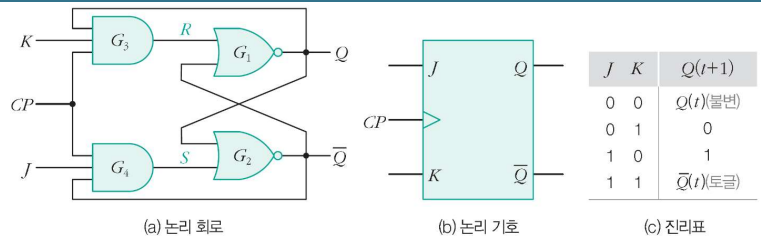


그림 3-71 JK 플립플롭의 구조

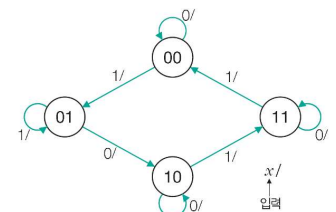


그림 3-81 순서 논리 회로에 대한 상태도

JK 플립플롭의 여기표

Q(t)	Q(t+1)	J	K
0	0	0	×
0	1	1	×
1	0	×	1
1	1	×	0

04 순서 논리 회로

● 간소화 방법을 이용해 출력 함수와 플립플롭의 입력 함수 유도

표 3-9 상태 여기표

현재 상태			입력		다음 상태		플립플롭 입력			
A	B	x	A	B	J _A	K _A	J _B	K _B		
0	0	0	0	0	0	x	0	x		
0	0	1	0	1	0	x	1	x		
0	1	0	1	0	1	x	x	1		
0	1	1	0	1	0	x	x	0		
1	0	0	1	0	x	0	0	x		
1	0	1	1	1	x	0	1	x		
1	1	0	1	1	x	0	x	0		
1	1	1	0	0	x	1	x	1		

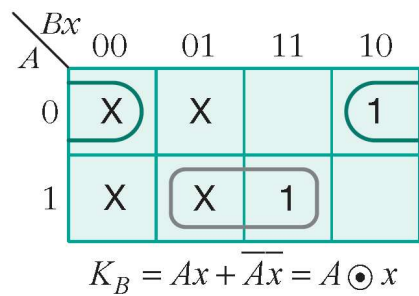
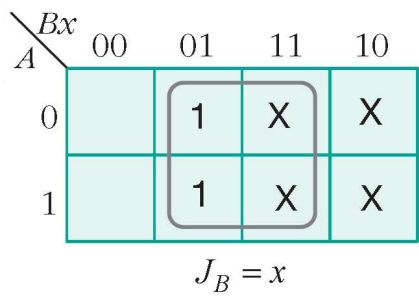
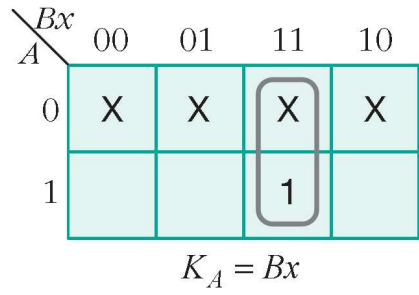
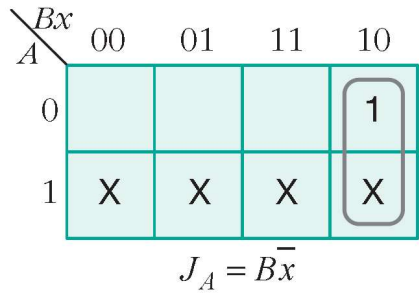


그림 3-82 카르노 맵을 이용한 간소화 과정

04 순서 논리 회로

● 순서 논리 회로도 작성

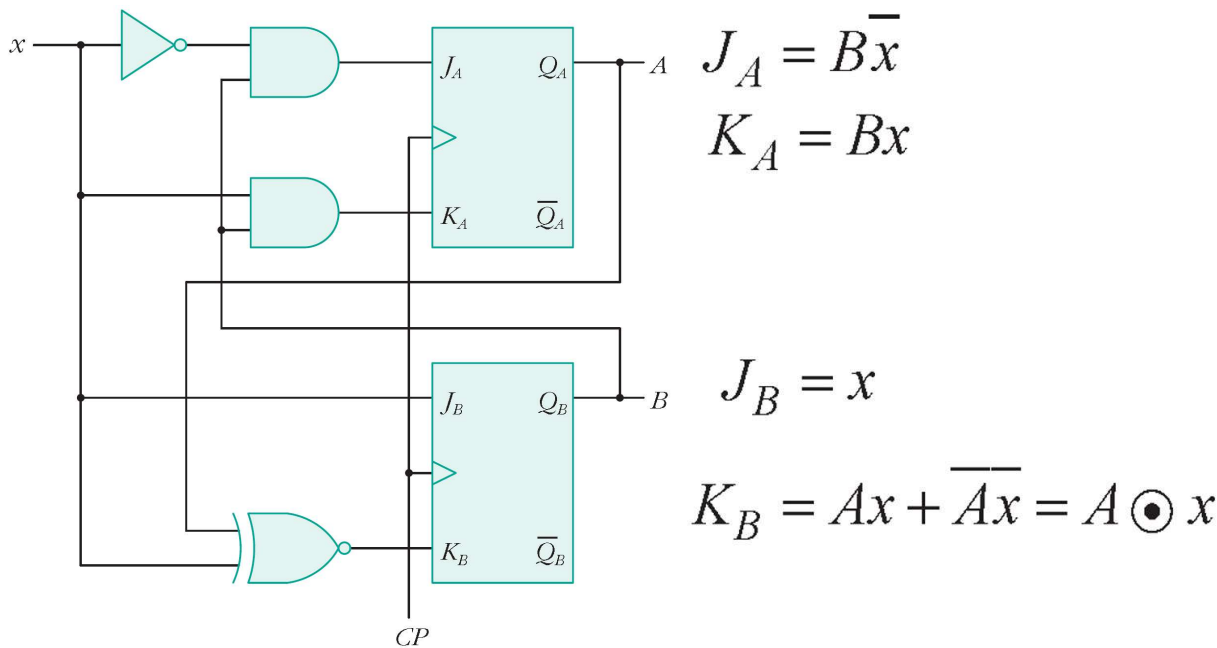


그림 3-83 순서 논리 회로의 구현

04 순서 논리 회로

□ 순서 논리 회로의 설계 과정

- ① 설계 사양으로부터 상태도와 상태표 작성
- ② 플립플롭의 수와 종류 결정
- ③ 플립플롭의 입력, 출력 및 각 상태에 문자 기호 부여
- ④ 상태표를 이용해 회로의 상태 여기표 작성
- ⑤ 간소화 방법을 이용해 출력 함수와 플립플롭의 입력 함수 유도
- ⑥ 순서 논리 회로도 작성

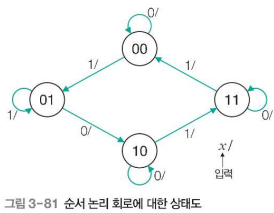


표 3-8 그림 3-81의 상태표

현재 상태		입력		다음 상태	
A	B	x		A	B
0	0	0		0	0
0	0	1		0	1
0	1	0		1	0
0	1	1		0	1
1	0	0		1	0
1	0	1		1	1
1	1	0		1	1
1	1	1		0	0

표 3-9 상태 여기표

현재 상태		입력		다음 상태		플립플롭 입력			
A	B	x		A	B	J_A	K_A	J_B	K_B
0	0	0		0	0	0	x	0	x
0	0	1		0	1	0	x	1	x
0	1	0		1	0	1	x	x	1
0	1	1		0	1	0	x	x	0
1	0	0		1	0	x	0	0	x
1	0	1		1	1	x	0	1	x
1	1	0		1	1	x	0	x	0
1	1	1		0	0	x	1	x	1

$Q(t)$	$Q(t+1)$	J	K
0	0	0	x
0	1	0	x
1	0	1	x
1	1	x	0

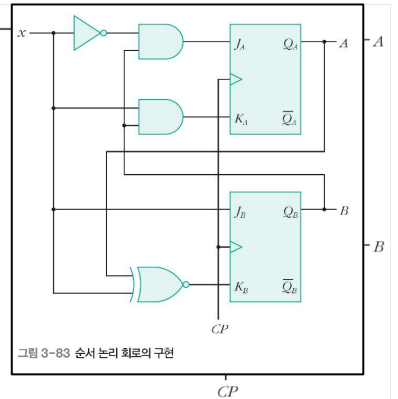


그림 3-83 순서 논리 회로의 구현

$Q(t)$	$Q(t+1)$	J	K
0	0	0	x
0	1	0	x
1	0	1	x
1	1	x	0

그림 3-82 카르노 맵을 이용한 간소화 과정

04 순서 논리 회로

4 카운터의 설계

- JK 플립플롭을 사용해 3비트 동기식 카운터를 순서 논리 회로 방식으로 설계해 보자.
- 상태 8가지(3비트), 동기식(클럭에 동기됨), 카운터(숫자를 증가→상향, 감소→하향)

상태도와 상태표 작성

상태표를 이용해 회로의 상태 여기표 작성

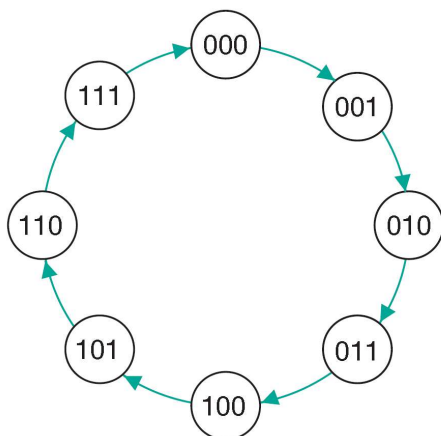


그림 3-84 3비트 동기식 2진 카운터의 상태도

표 3-10 3비트 동기식 2진 카운터의 상태 여기표

현재 상태			다음 상태			플립플롭 입력					
Q_C	Q_B	Q_A	Q_C	Q_B	Q_A	J_C	K_C	J_B	K_B	J_A	K_A
0	0	0	0	0	1	0	x	0	x	1	x
0	0	1	0	1	0	0	x	1	x	x	1
0	1	0	0	1	1	0	x	x	0	1	x
0	1	1	1	0	0	1	x	x	1	x	1
1	0	0	1	0	1	x	0	0	x	1	x
1	0	1	1	1	0	x	0	1	x	x	1
1	1	0	1	1	1	x	0	x	0	1	x
1	1	1	0	0	0	x	1	x	1	x	1

JK 플립플롭의 여기표

$Q(t)$	$Q(t+1)$	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

04 순서 논리 회로

간소화 방법을 이용해 플립플롭의 입력 함수 유도

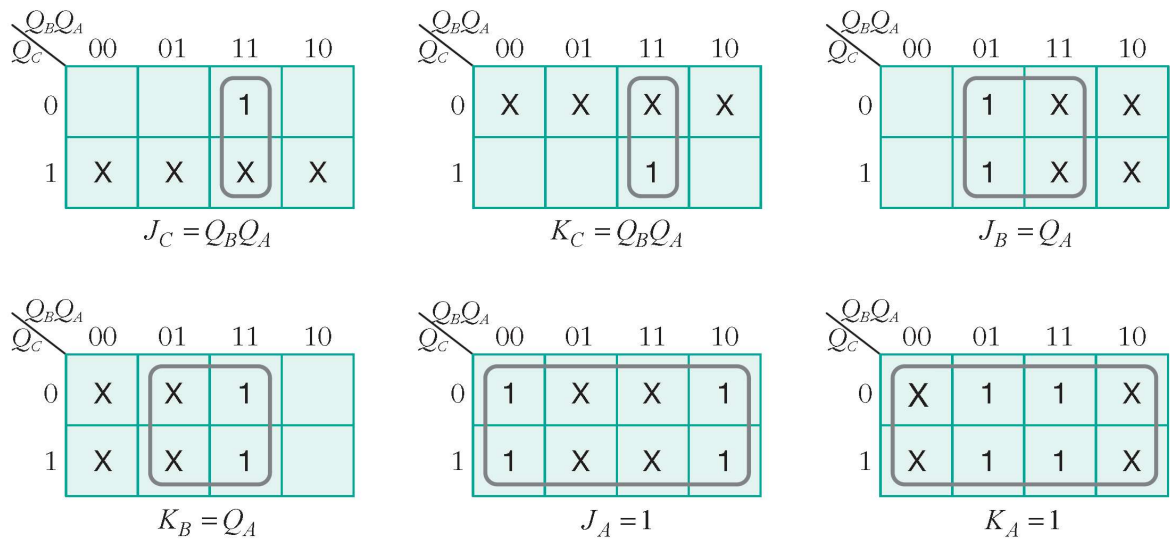


그림 3-85 3비트 동기식 2진 카운터의 카르노 맵을 이용한 간소화 과정

04 순서 논리 회로

순서 논리 회로도 작성

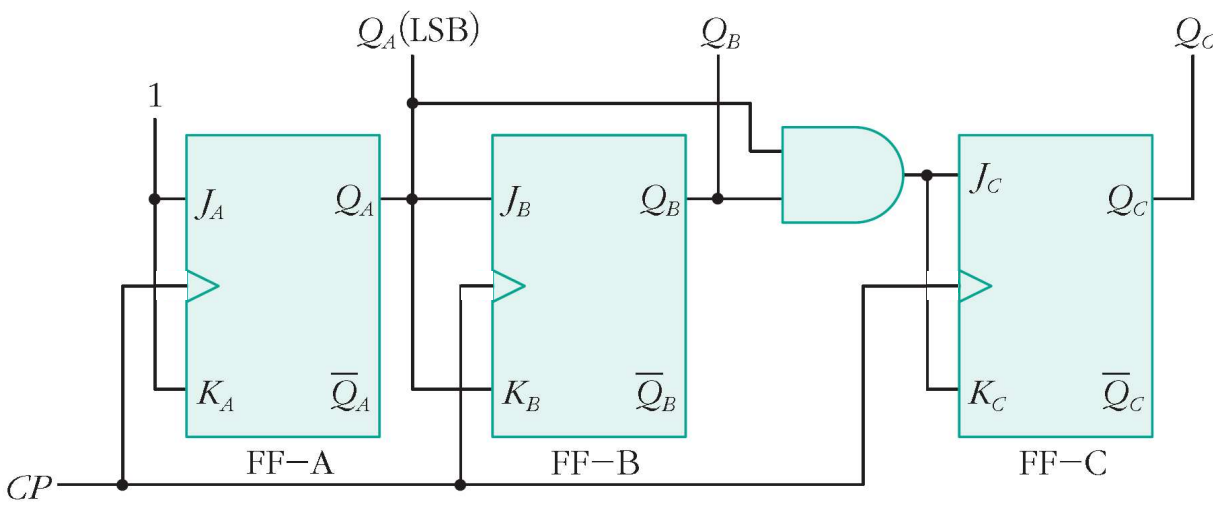
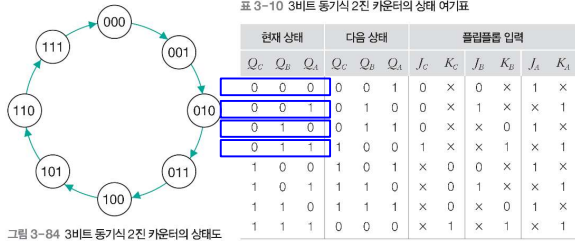


그림 3-86 3비트 동기식 2진 카운터 회로도

04 순서 논리 회로



J	K	$Q(t+1)$
0	0	$Q(t)$ (불변)
0	1	0
1	0	1
1	1	$\bar{Q}(t)$ (토글)

(c) 진리표

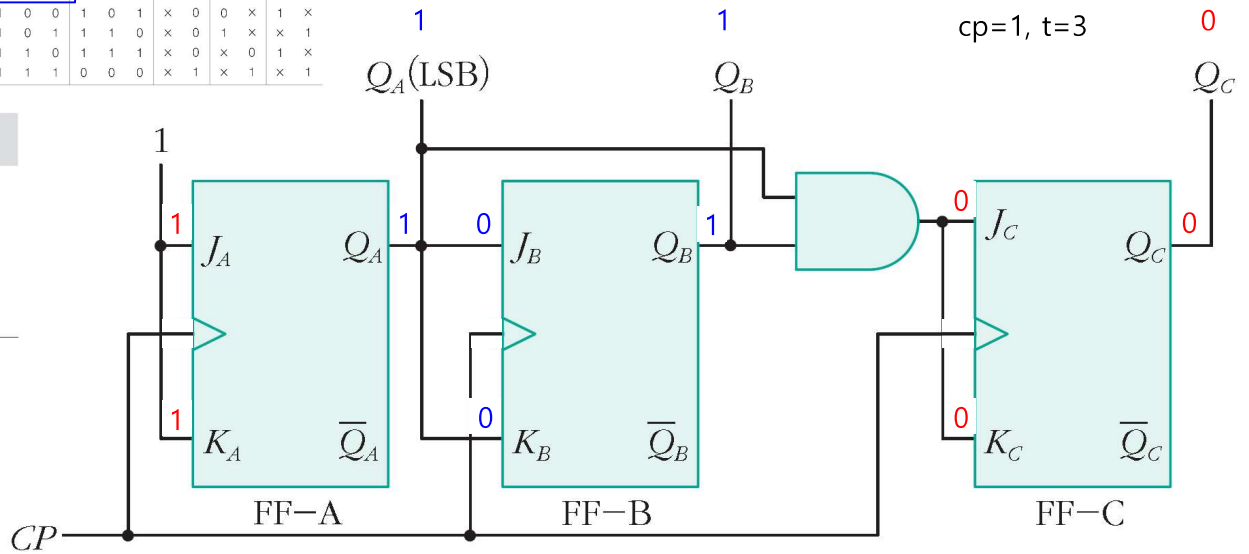
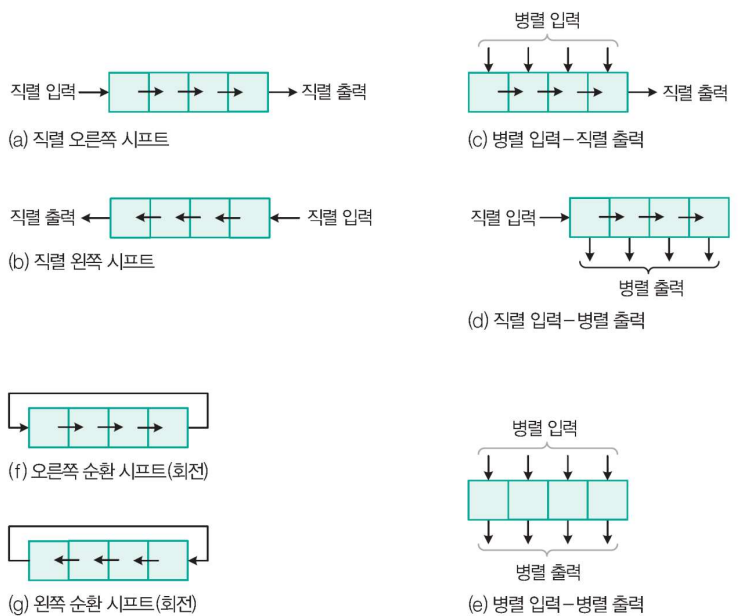


그림 3-86 3비트 동기식 2진 카운터 회로도

04 순서 논리 회로

5 레지스터

- 레지스터(register)는 기본적으로 데이터 비트를 저장하는 소자
- 대부분의 레지스터에서는 D 플립플롭이 사용되며 각 D 플립플롭에 한 비트씩 저장
- 따라서 n 비트 레지스터는 플립플롭 n 개로 구성되며, n 비트의 2진 정보를 저장할 수 있다.



04 순서 논리 회로

□ 4비트 레지스터

- 공통 클럭 신호의 상승 에지에서 입력 데이터(I_A, I_B, I_C, I_D)가 D 플립플롭 4개에 동시에 저장되며, 출력(O_A, O_B, O_C, O_D)에서는 언제든지 저장된 데이터를 출력할 수 있다.
- $\overline{CLR} = 0$ 이면 클럭에 관계없이 언제든지 모든 플립플롭의 출력을 0으로 만들 수 있다.

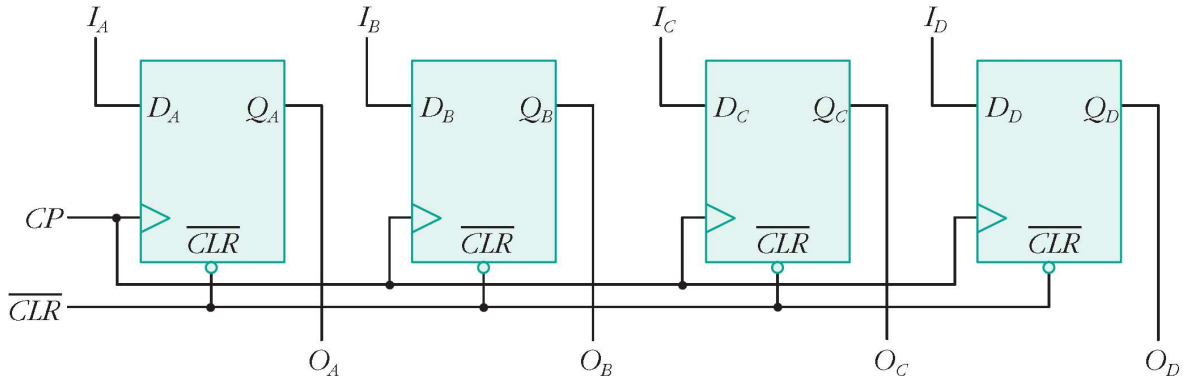


그림 3-88 4비트 레지스터

04 순서 논리 회로

□ 시프트 레지스터

- 클럭 펄스가 입력될 때마다 클럭 펄스의 상승 에지에서 입력 데이터가 한 비트씩 오른쪽으로 시프트하면서 저장($I \rightarrow Q_A, Q_A \rightarrow Q_B, Q_B \rightarrow Q_C, Q_C \rightarrow Q_D$)
- 이 과정은 새로운 클럭 펄스의 상승 에지마다 반복되므로 네 번째 클럭 펄스의 상승 에지에서 처음에 입력된 데이터 비트가 Q_D 에 나타난다.

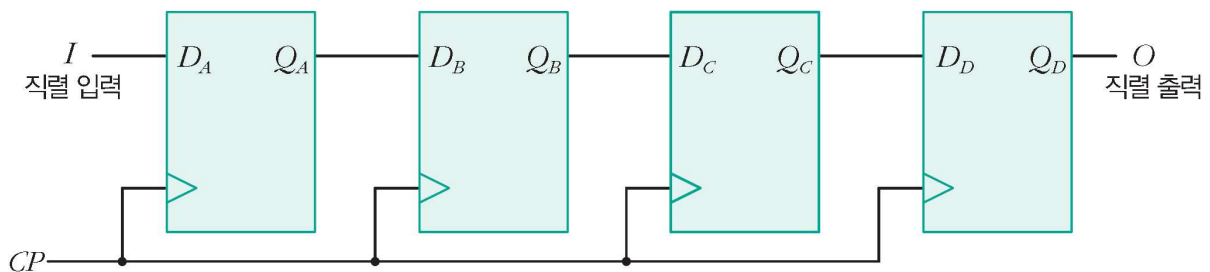


그림 3-89 4비트 시프트 레지스터(직렬 입력 - 직렬 출력)

▣ 병렬 로드를 가지는 양방향 시프트 레지스터

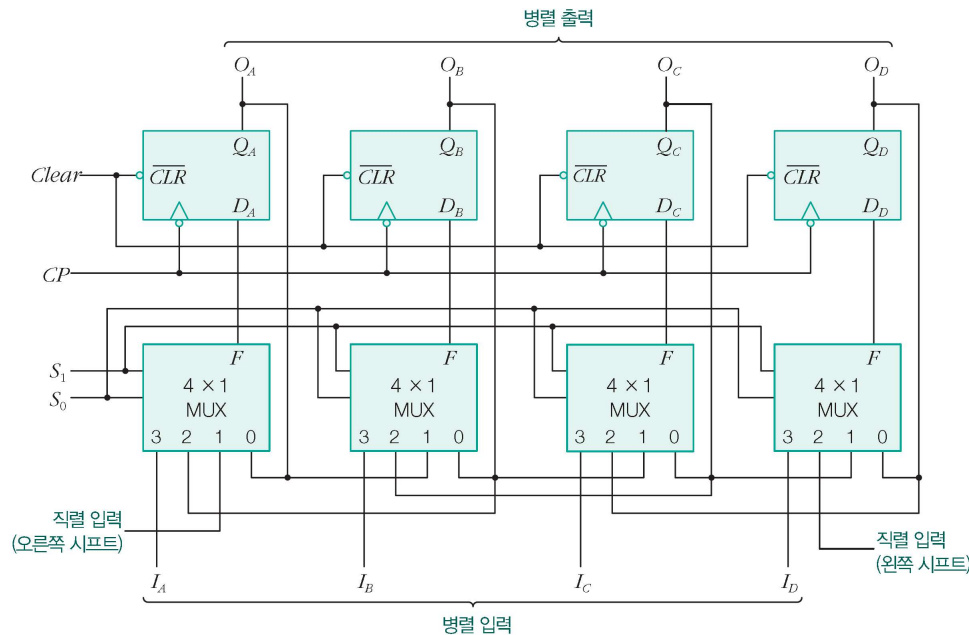


그림 3-90 병렬 로드를 가지는 4비트 양방향 시프트 레지스터

▣ 병렬 로드를 가지는 양방향 시프트 레지스터(계속)

표 3-11 양방향 시프트 레지스터의 제어표

모드 제어		레지스터 동작	설명
S ₁	S ₀		
0	0	불변 상태가 된다.	각 멀티플렉서의 입력 채널 00이 선택되어 레지스터는 현재 출력 값이 다시 플립플롭 D 입력에 공급되어 클록 펄스가 입력되어도 현재 상태를 유지한다.
0	1	오른쪽 시프트가 수행된다.	각 멀티플렉서의 입력 채널 10이 선택되어 각 플립플롭의 출력 Q는 오른쪽 플립플롭의 D 입력에 연결되어 오른쪽 시프트를 수행하며, 맨 왼쪽 플립플롭의 D 입력에는 직렬 데이터가 입력된다.
1	0	왼쪽 시프트가 수행된다	각 멀티플렉서 입력 채널 2가 선택되어 각 플립플롭의 출력 Q는 왼쪽 플립플롭의 D 입력에 연결되어 왼쪽 시프트를 수행하며, 맨 오른쪽 플립플롭의 D 입력에는 직렬 데이터가 입력된다.
1	1	병렬 입력이 수행된다.	각 멀티플렉서 입력 채널 3이 선택되어 병렬 입력(I _D ~I _A)의 2진 데이터는 클록 펄스가 입력될 때 레지스터에 로드된다.